Insurance Advisor Project Report

Full source code bundle, test data, and application screenshots for profile creation, document upload, chat, and recommendation flows.

1. Project Definition

The Insurance Advisor is an intelligent application designed to help users understand and navigate their insurance policies. It combines artificial intelligence, natural language processing, and document analysis to provide personalized insurance guidance. The platform acts as an AI-powered assistant that can interpret complex insurance documents and answer user queries in plain language.

2. Project Purpose

Primary Goals:
• Simplify insurance policy comprehension for non-technical users
• Enable users to quickly find relevant information from complex policy documents
• Provide AI-driven personalized insurance recommendations
• Create an interactive conversational interface for insurance queries
• Reduce confusion and empower informed insurance decisions

Key Benefits:
• User-friendly interface for policy document analysis
• Real-time AI responses to insurance questions
• Personalized recommendations based on user profile and documents
• Secure document upload and processing
• Conversation history tracking for reference

3. Technology Stack

Frontend & Framework:
• Streamlit - Interactive web application framework for rapid UI development

Backend & APIs:
• Python 3.x - Core programming language
• OpenAI API - Large Language Model (LLM) for AI responses and analysis
• LangChain - Framework for LLM orchestration and chaining

Data Processing & Storage:
• LangChain Vector Stores - For document embeddings and semantic search
• FAISS (Facebook AI Similarity Search) - Efficient similarity search for retrieved documents
• Pydantic - Data validation and serialization

Document Processing:
• PDF/Text parsing and preprocessing capabilities
• Document chunking for optimal vector store ingestion

State Management:
• Streamlit Session State - For maintaining conversation history and user data
• In-memory memory management for efficient state tracking

Testing & Quality Assurance:
• Python unittest framework - Unit testing
• pytest - Advanced testing capabilities

Report Generation:

• ReportLab - PDF generation and document creation

4. Use Cases & Applications

Individual Users:
• Understand personal health insurance policy coverage and limitations
• Get answers to specific policy questions without contacting customer service
• Receive recommendations for policy optimization based on personal profile
• Clarify claim procedures and coverage scenarios

Insurance Agents & Brokers:
• Quickly explain policy details to clients
• Compare policies and provide recommendations
• Assist in policy selection process
• Improve customer service efficiency

Enterprises & Insurance Companies:
• Provide first-tier customer support for policy inquiries
• Reduce customer support ticket volume
• Improve customer satisfaction and engagement
• Support policy literacy and education initiatives

Healthcare & Benefits Administration:
• Help employees understand company health benefits
• Reduce HR department inquiries
• Support employee benefits onboarding
• Answer FAQs about coverage and enrollment

5. Application Workflow & Screenshots

The Insurance Advisor follows a structured user journey designed for intuitive navigation and maximum value extraction from insurance documents. The application guides users through five primary stages:

Stage 1: Profile Creation
The user journey begins with profile creation, where users enter personal information and insurance preferences. This stage establishes the foundation for personalized recommendations. The system validates input and confirms successful profile setup, preparing the database for document storage and analysis.

Stage 2: Document Upload
Users securely upload their insurance policy documents in supported formats. The system processes documents through a preprocessing pipeline that extracts text, segments content into meaningful chunks, and generates semantic embeddings. This preparation enables fast, accurate document retrieval when answering user queries.

Stage 3: Interactive Chat
With documents uploaded and indexed, users can initiate conversations with the AI assistant. Each query triggers a RAG (Retrieval-Augmented Generation) process: the system searches vector embeddings to find relevant policy sections and feeds this context to the LLM, ensuring responses are grounded in actual policy text and not hallucinations.

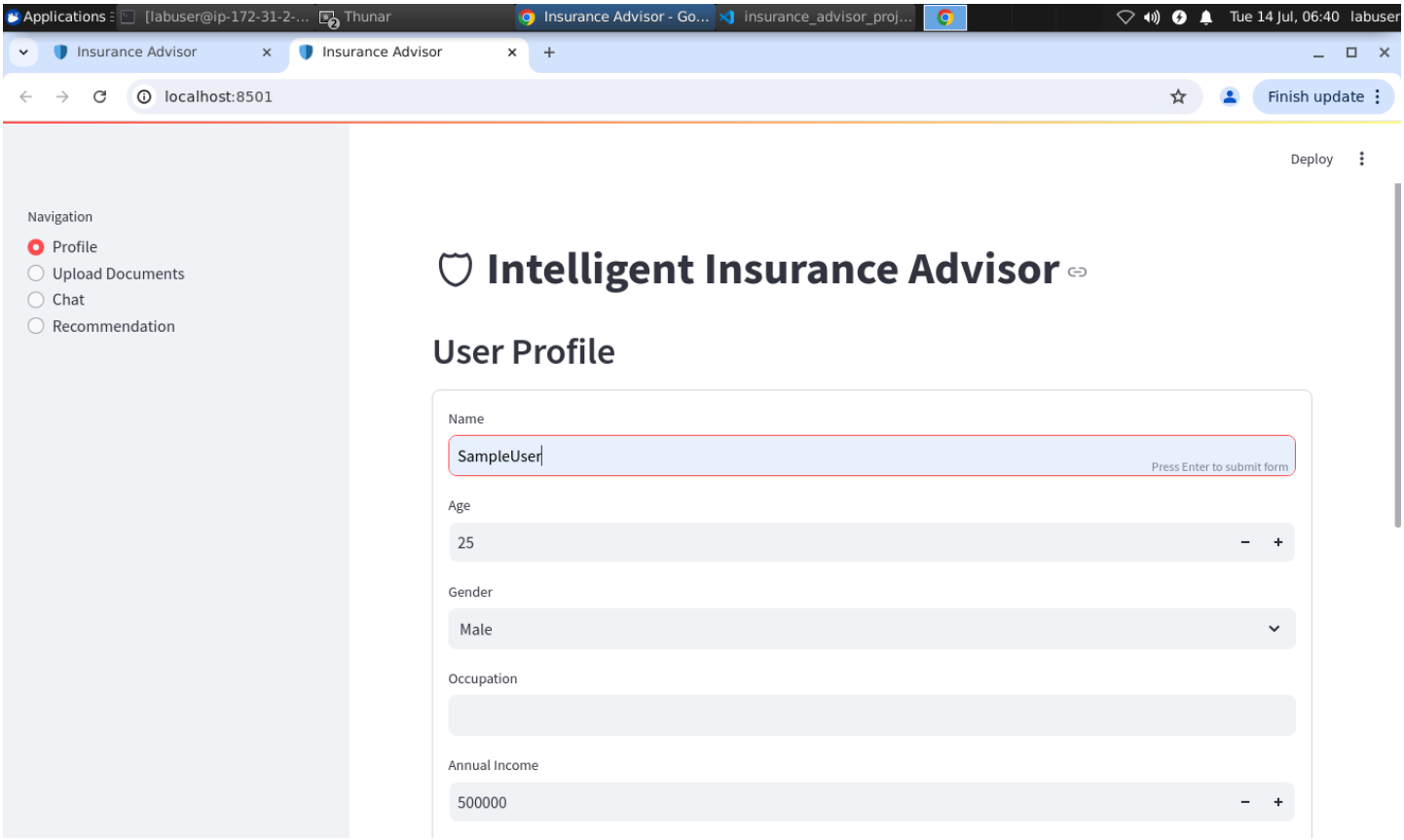
Stage 4: Conversation Management
The system maintains complete conversation history, allowing users to review previous questions and responses. This context is also fed to the LLM for multi-turn conversations, enabling natural dialogue that remembers prior context and builds on previous answers.

Stage 5: Smart Recommendations
Based on the user profile, uploaded documents, and conversation patterns, the recommendation engine generates personalized insurance suggestions. These recommendations consider coverage gaps, policy optimization opportunities, and the user's stated preferences and concerns discovered through chat interactions.

Application Walkthrough: Screenshots & Features

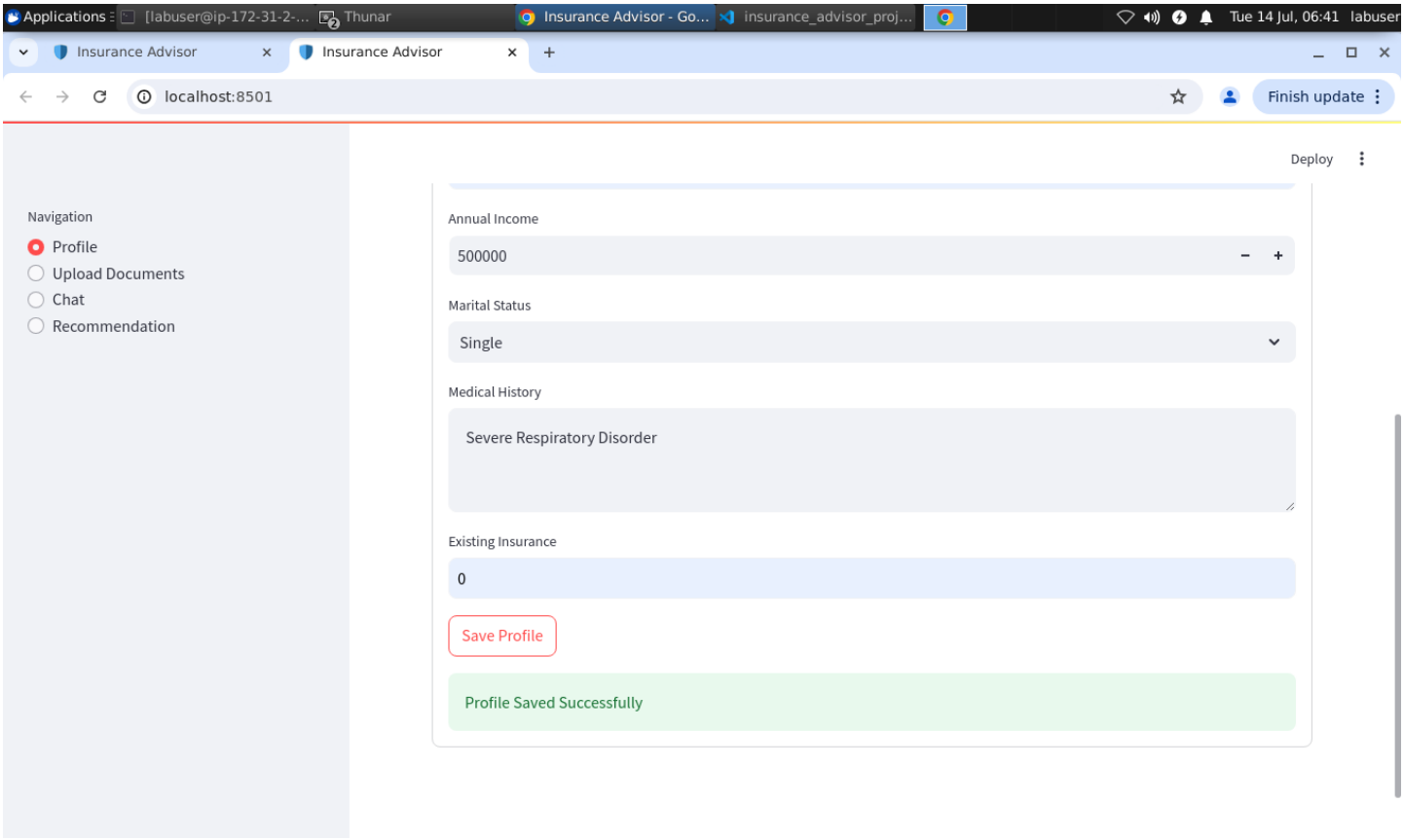
The following screenshots showcase each stage of the Insurance Advisor workflow in action:

User Profile Creation



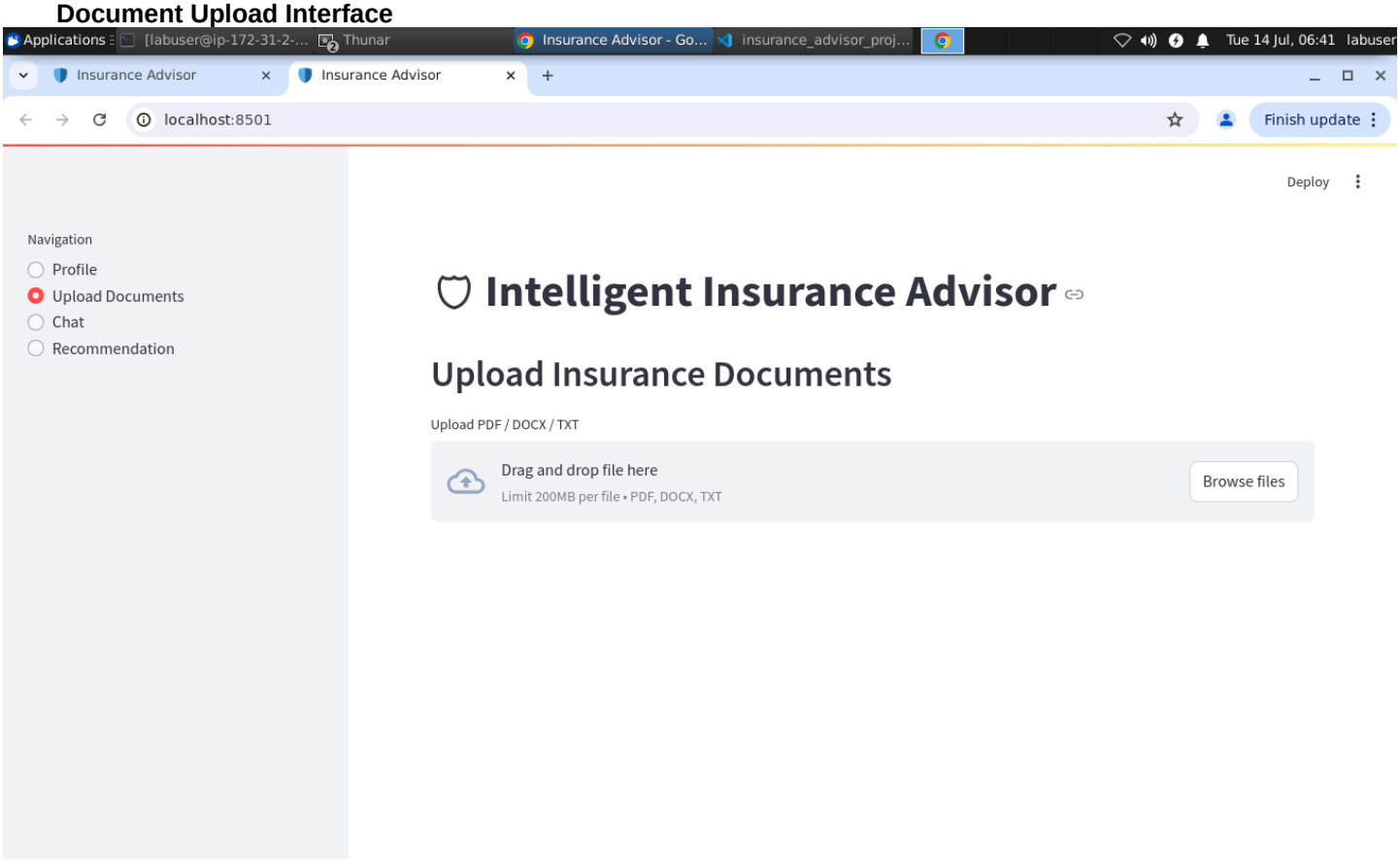
Users begin by creating a profile with their personal details and insurance preferences.

Profile Creation Successful



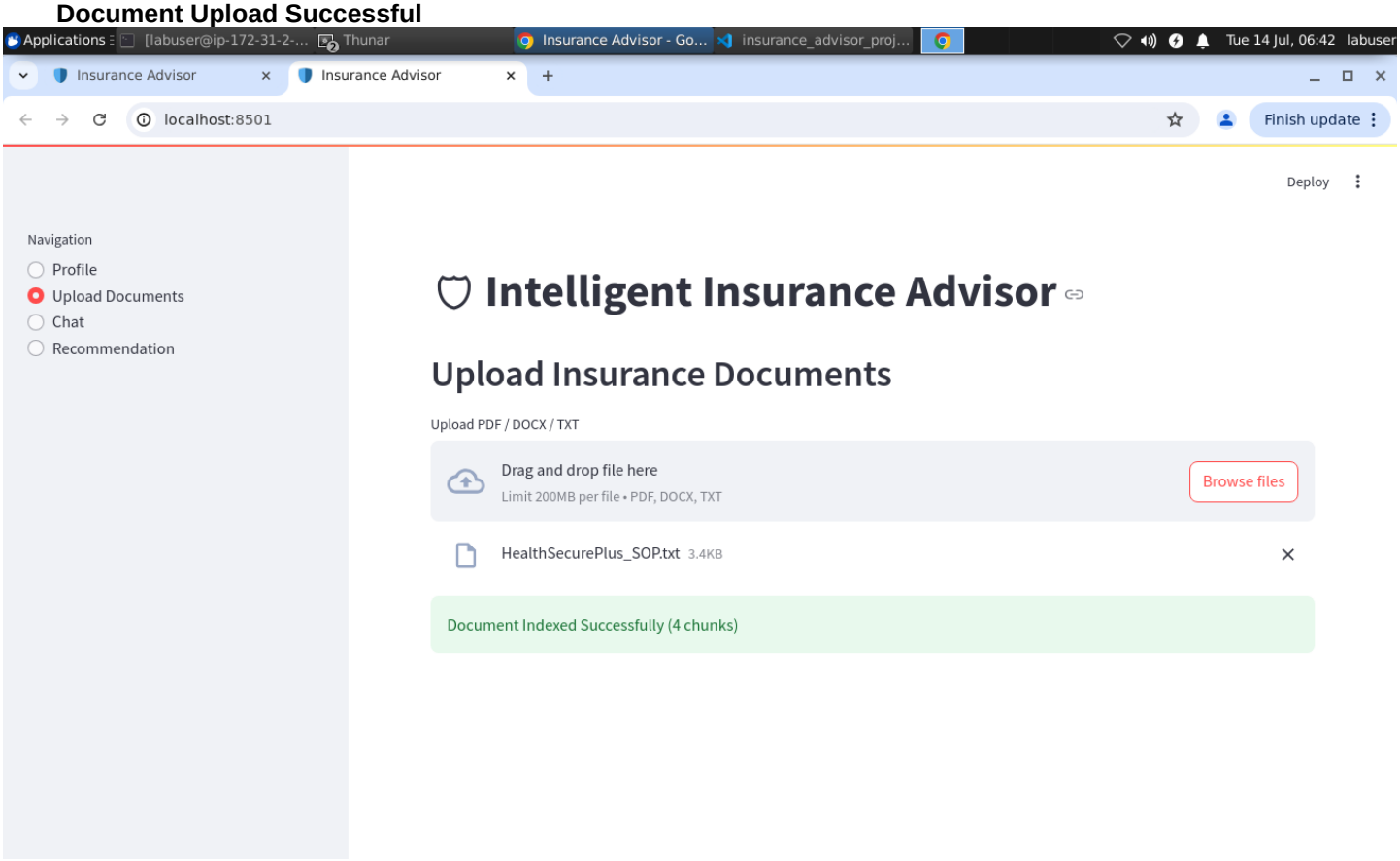
Confirmation screen after successful profile setup.

Document Upload Interface



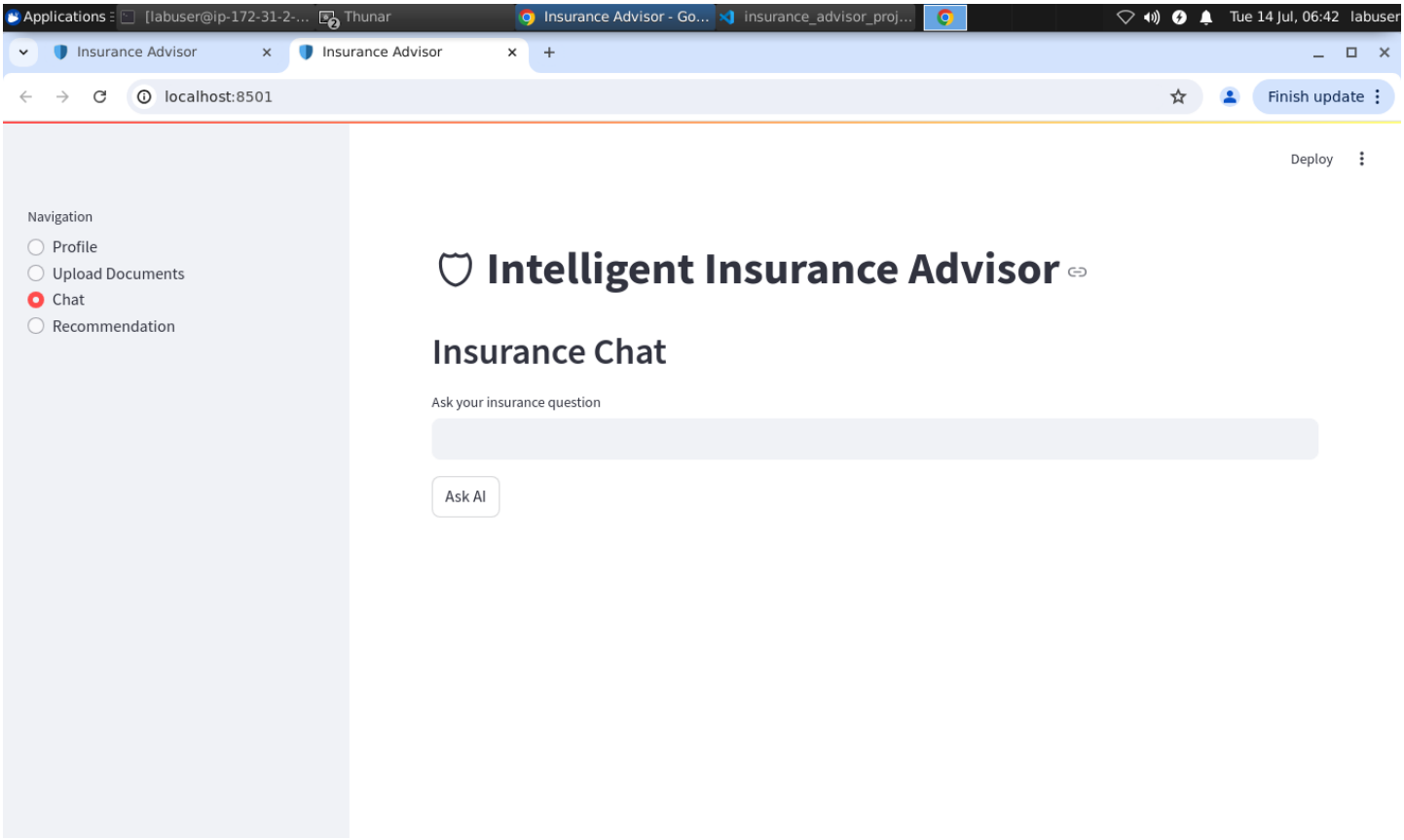
The document upload interface allows users to upload insurance policy documents for analysis.

Document Upload Successful



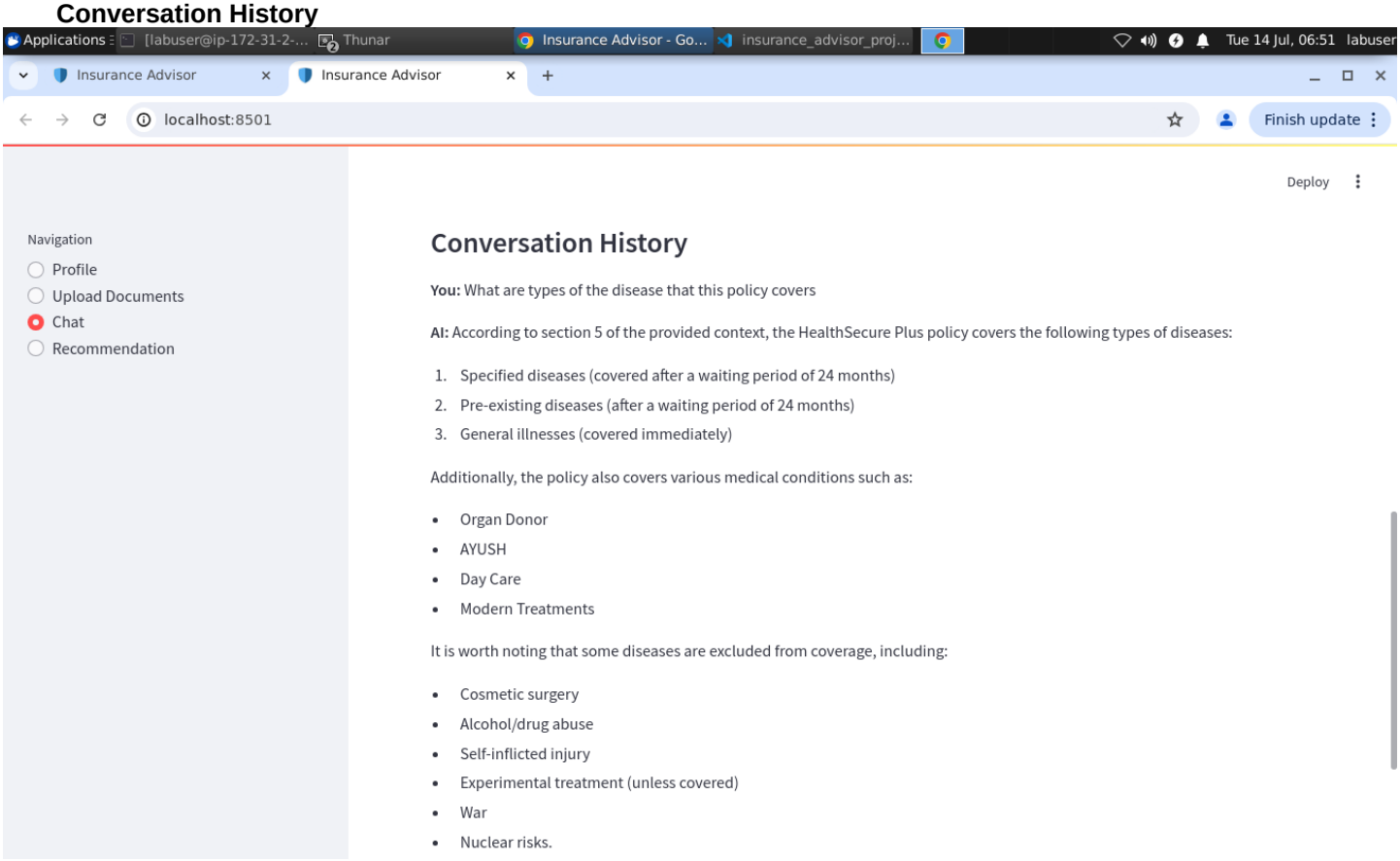
Confirmation of successful document upload to the system.

Chat Screen



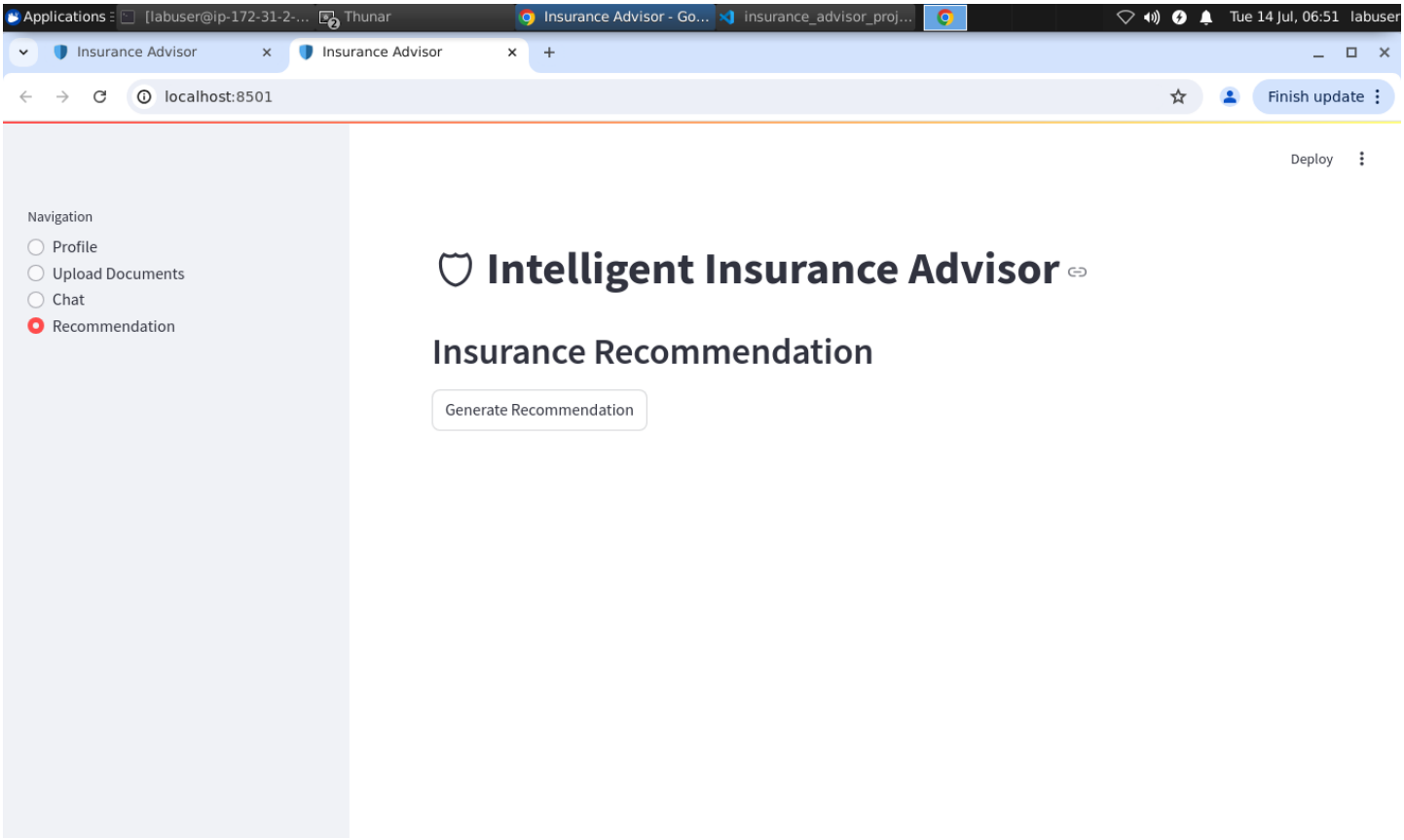
Interactive chat interface where users can ask questions about their insurance policies and documents.

Conversation History



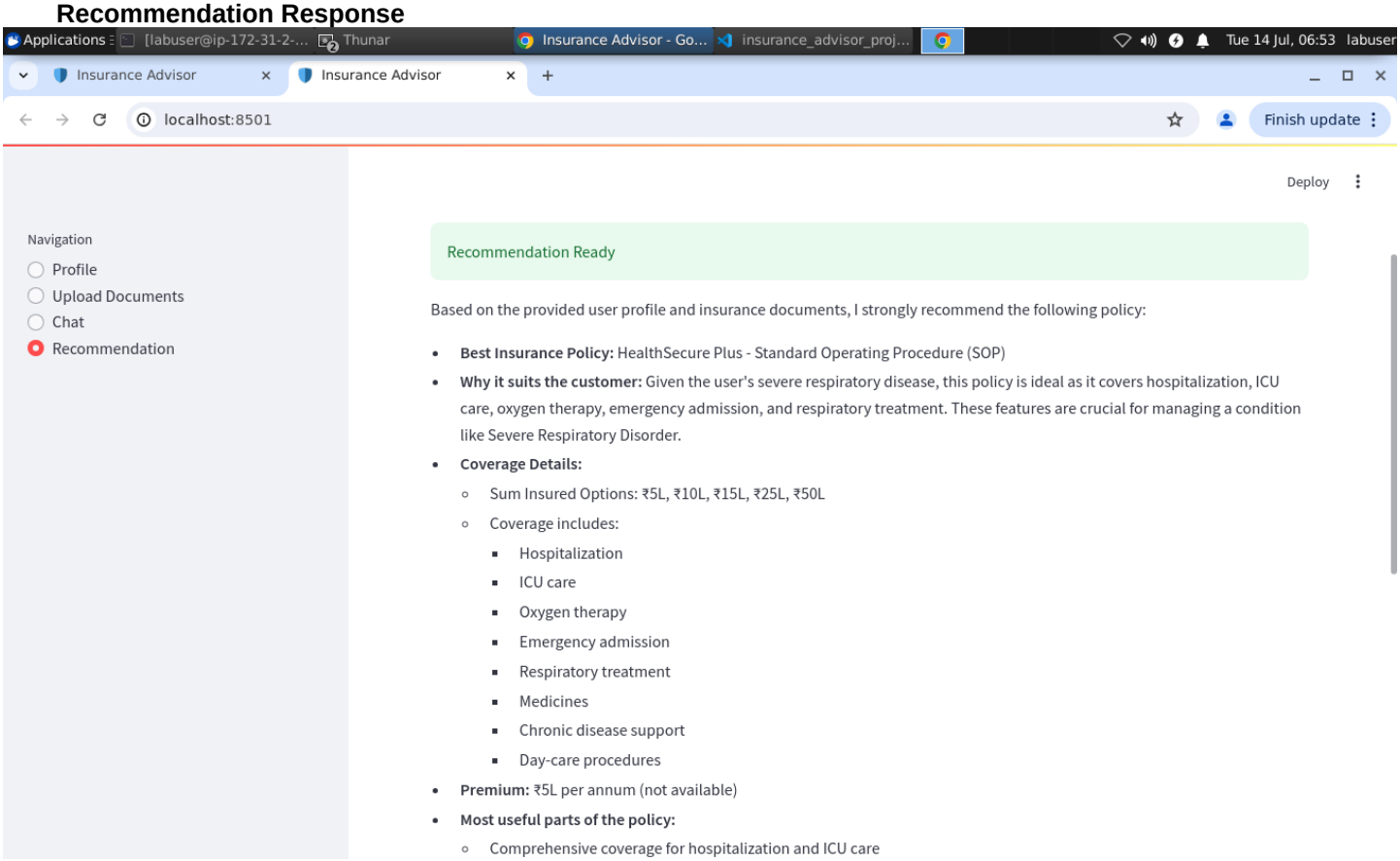
View of the complete conversation history showing user queries and AI responses.

Recommendation View



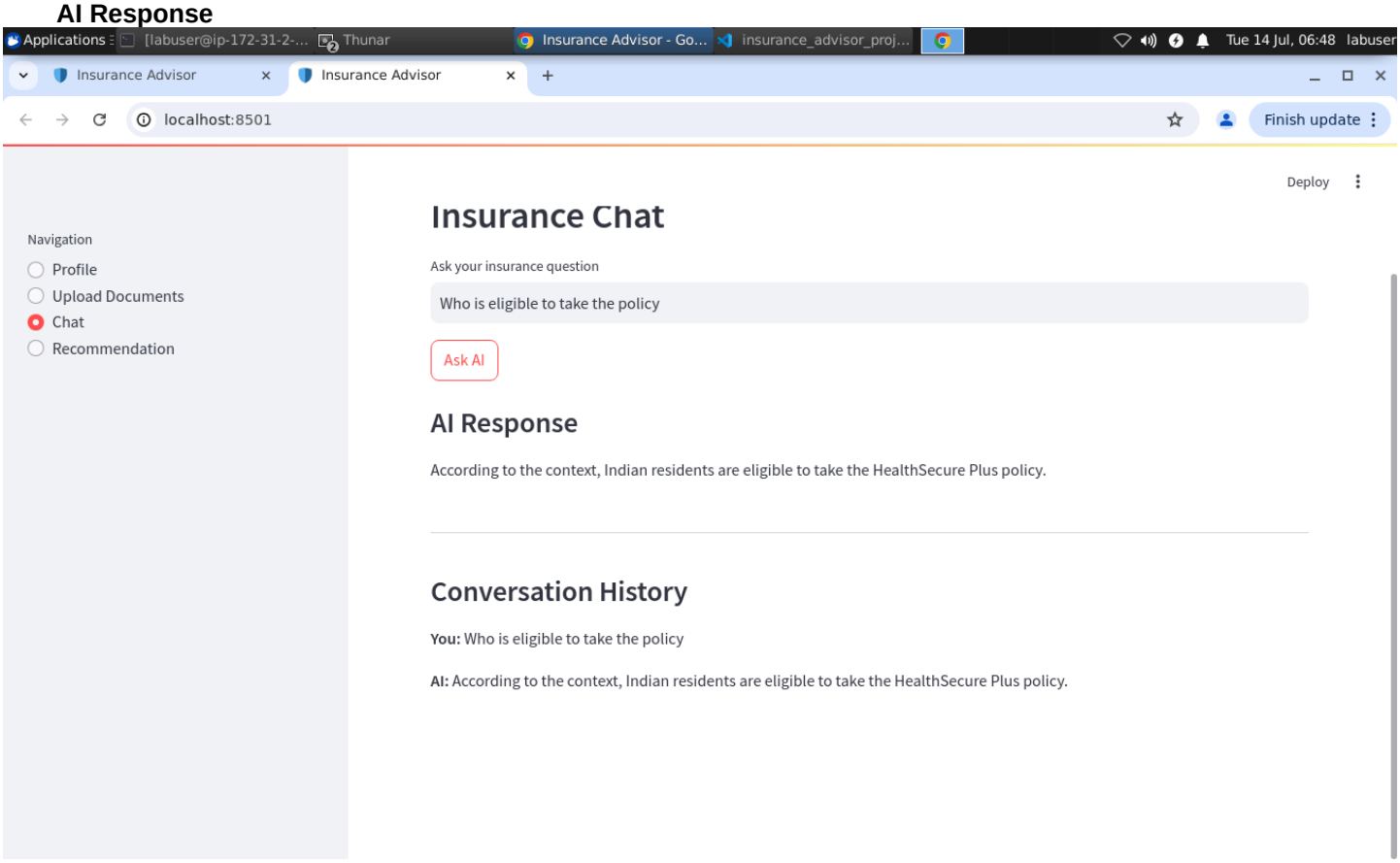
The recommendation engine analyzes documents and provides personalized insurance recommendations.

Recommendation Response



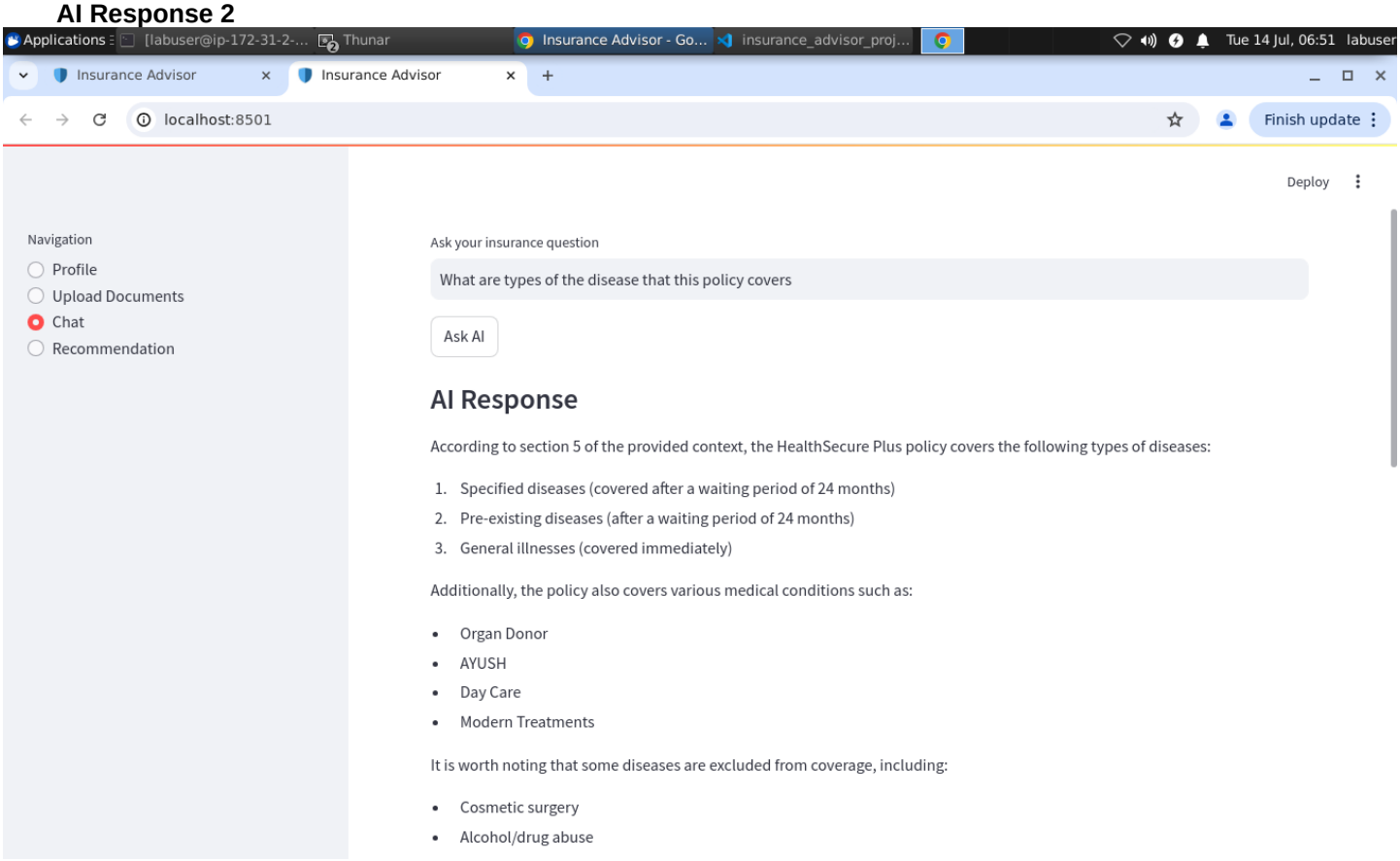
Detailed recommendations tailored to the user's profile and uploaded documents.

AI Response



Sample AI-generated response to user insurance queries.

AI Response 2

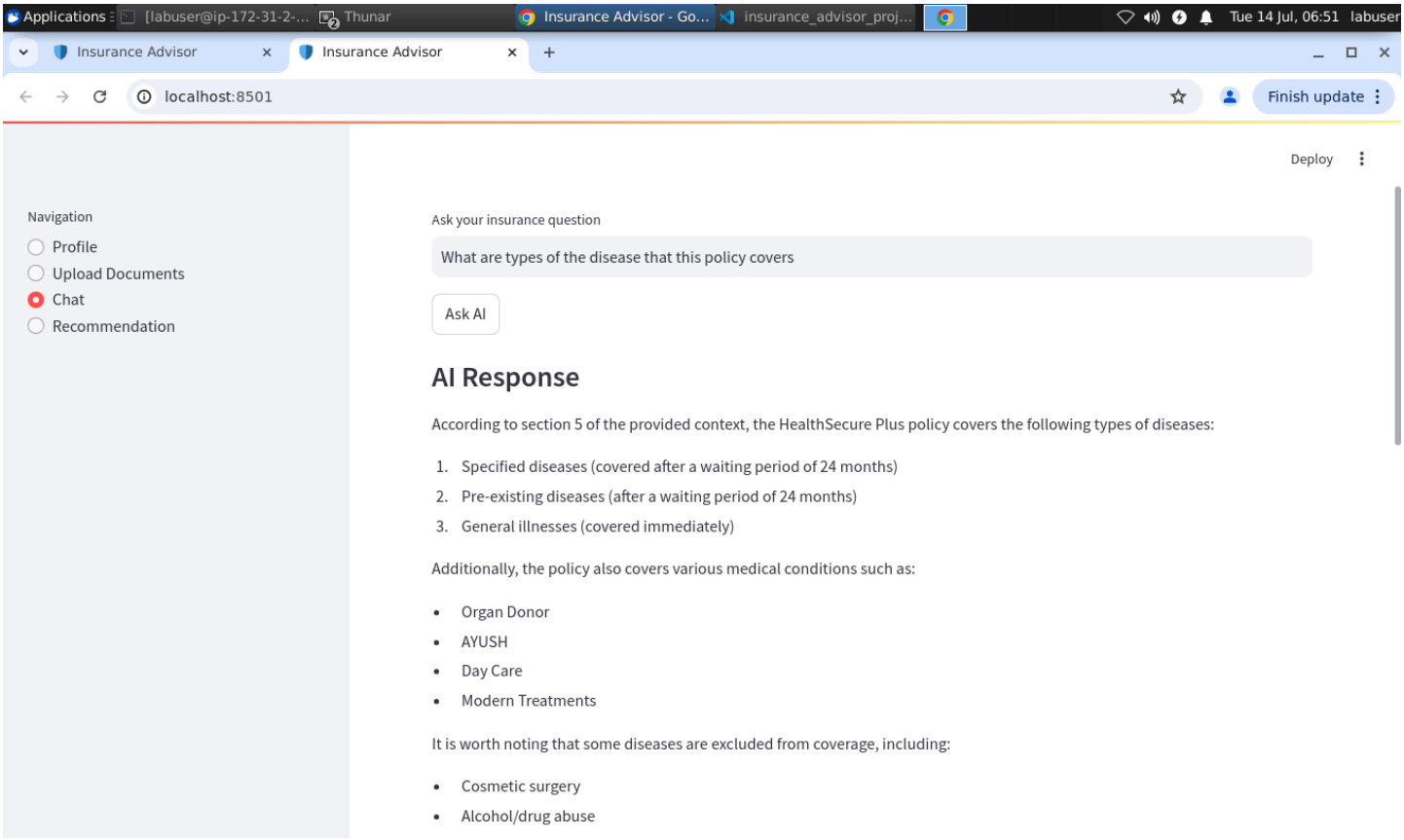


Additional example of AI analysis and recommendations.

6. Test Data & Validation

This section includes a unittest-style validation screenshot showing the app with sample insurance test data.

Test Data View



7. Core Features & Architecture

The Insurance Advisor implements a modular architecture with the following key components:

User Management: User profiles store personal information and insurance preferences for personalized recommendations.

Document Upload & Processing: Secure upload and preprocessing of insurance policy documents with support for multiple formats.

Retrieval-Augmented Generation (RAG): Combines document embeddings with LLM capabilities to retrieve relevant policy sections and generate accurate responses.

Conversational AI: Maintains conversation history and context for natural multi-turn dialogue.

Smart Recommendations: Analyzes user profile and documents to provide personalized insurance recommendations.

Workflow Integration: Graph-based workflow system for orchestrating complex multi-step processes.

8. Source Code & Implementation

The following sections contain the complete source code and configuration files for the Insurance Advisor project.

Main Streamlit App

Source file: streamlit_app.py

```
import os
import streamlit as st

from memory import memory
from rag import build_vector_store, get_context
from recommendation import generate_recommendation
from llm import ask_llama
from graph import graph

UPLOAD_FOLDER = "uploads"

os.makedirs(UPLOAD_FOLDER, exist_ok=True)

st.set_page_config(
page_title="Insurance Advisor",
page_icon="■■",
layout="wide"
)

if "profile" not in st.session_state:
st.session_state.profile = {}
if "chat_history" not in st.session_state:
st.session_state.chat_history = []
if "vector_store" not in st.session_state:
st.session_state.vector_store = None
if "documents" not in st.session_state:
st.session_state.documents = []

memory.profile = st.session_state.profile
memory.chat_history = st.session_state.chat_history
memory.vector_store = st.session_state.vector_store
memory.documents = st.session_state.documents

st.title("■■ Intelligent Insurance Advisor")

menu = st.sidebar.radio(
"Navigation",
[
"Profile",
"Upload Documents",
"Chat",
"Recommendation"
]
)

############################################################
# PROFILE
############################################################

if menu == "Profile":

st.header("User Profile")

with st.form("profile_form"):

name = st.text_input("Name")

age = st.number_input(
"Age",
18,
100,
25
)
```

```
gender = st.selectbox(
"Gender",
[
"Male",
"Female",
"Other"
]
)

occupation = st.text_input("Occupation")

income = st.number_input(
"Annual Income",
100000,
50000000,
500000
)

marital = st.selectbox(
"Marital Status",
[
"Single",
"Married"
]
)

medical = st.text_area(
"Medical History"
)

existing = st.text_input(
"Existing Insurance"
)

submit = st.form_submit_button(
"Save Profile"
)

if submit:
profile_data = {
"name": name,
"age": age,
"gender": gender,
"occupation": occupation,
"income": income,
"marital_status": marital,
"medical_history": medical,
"existing_policy": existing
}

memory.profile = profile_data
st.session_state.profile = profile_data

st.success("Profile Saved Successfully")
############################################################
# DOCUMENT UPLOAD
############################################################

elif menu == "Upload Documents":

st.header("Upload Insurance Documents")

uploaded = st.file_uploader(
"Upload PDF / DOCX / TXT",
type=[
"pdf",
"docx",
"txt"
]
)

if uploaded:
```

```
filepath = os.path.join(
UPLOAD_FOLDER,
uploaded.name
)

with open(filepath, "wb") as f:
f.write(uploaded.read())

with st.spinner("Processing Document..."):
chunks = build_vector_store(filepath)

st.session_state.vector_store = memory.vector_store
st.session_state.documents = memory.documents

st.success(
f"Document Indexed Successfully ({chunks} chunks)"
)
############################################################
# CHAT
############################################################

elif menu == "Chat":

st.header("Insurance Chat")

question = st.text_input(
"Ask your insurance question"
)

if st.button("Ask AI"):

if memory.vector_store is None:

st.warning(
"Please upload an insurance document first."
)

else:

context = get_context(question)
answer = ask_llama(context, question)

memory.chat_history.append(
(
question,
answer
)
)

st.markdown("### AI Response")
st.write(answer)

if memory.chat_history:

st.divider()

st.subheader("Conversation History")

for q, a in reversed(memory.chat_history):

st.markdown(f"**You:** {q}")

st.markdown(f"**AI:** {a}")

############################################################
# RECOMMENDATION
############################################################

elif menu == "Recommendation":

st.header("Insurance Recommendation")

if memory.vector_store is None:
```

```
st.info(
"Upload insurance documents first."
)

elif not memory.profile:

st.info(
"Complete your profile first."
)

else:

if st.button("Generate Recommendation"):
context = get_context(
"Recommend the best insurance policy based on the user profile and uploaded documents."
)
answer = generate_recommendation(memory.profile, context)

st.success("Recommendation Ready")
st.write(answer)
```

LLM Connector

Source file: llm.py

```
"""LLM helpers for invoking the local Ollama-backed LLM.

This module wraps the `ChatOllama` client and exposes a simple
`ask_llama(context, question)` helper used by the Streamlit app.
"""

from langchain_community.chat_models import ChatOllama
from dotenv import load_dotenv
import os

load_dotenv()

llm = ChatOllama(
model=os.getenv("LLAMA_MODEL"),
temperature=0.2
)


def ask_llama(context: str, question: str) -> str:
"""Send a prompt to the LLM that includes the provided context.

The function builds a constrained prompt that instructs the model
to answer using only the supplied context. Returns the raw text
content of the model response.
"""
prompt = f"""
You are an Insurance Advisor :
Answer only using the provided context.
Context:
{context}
Question:
{question}
"""

response = llm.invoke(prompt)
return response.content
```

RAG / Vector Store

Source file: rag.py

```
"""rag.py

Utilities for loading documents, building an in-memory vector store
```

```
and retrieving contextual text for a user question. This module is
used by the Streamlit app to implement a simple RAG (retrieval-augmented
generation) workflow.

Functions:
- load_document(file_path): load text from PDF/DOCX/TXT
- build_vector_store(file_path): split and store embeddings in memory
- retrieve(question): perform similarity search against the in-memory store
- get_context(question): return joined page content for the top documents
"""

from pathlib import Path
from langchain_community.document_loaders import (PyPDFLoader,TextLoader,Docx2txtLoader)

from langchain.text_splitter import RecursiveCharacterTextSplitter

from langchain_core.vectorstores import InMemoryVectorStore
from dotenv import load_dotenv
from langchain_community.embeddings import OllamaEmbeddings
import os
from memory import memory

load_dotenv()

embedding_model=OllamaEmbeddings(model=os.getenv("EMBEDDING_MODEL"))

def load_document(file_path: str):
"""Load a document from disk and return LangChain document objects.

Supports PDF, DOCX and TXT based on file suffix. Raises an
Exception for unsupported types.
"""
suffix = Path(file_path).suffix.lower()
if suffix == ".pdf":
loader = PyPDFLoader(file_path)
elif suffix == ".docx":
loader = Docx2txtLoader(file_path)
elif suffix == ".txt":
loader = TextLoader(file_path)
else:
raise Exception("Unsupported file type")

return loader.load()


def build_vector_store(file_path: str):
"""Create an in-memory vector store from a document file.

Splits the document into chunks, computes embeddings and stores them
in `memory.vector_store`. Returns the number of chunks created.
"""
documents = load_document(file_path)
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(documents)
vector_store = InMemoryVectorStore(embedding=embedding_model)
vector_store.add_documents(chunks)
memory.vector_store = vector_store
memory.documents = chunks
return len(chunks)


def retrieve(question: str, k: int = 4):
"""Return the top-k similar document chunks for a question.

If no vector store exists yet, returns an empty list.
"""
if memory.vector_store is None:
return []

docs = memory.vector_store.similarity_search(question, k=k)
return docs
```

```
def get_context(question: str):
"""Build a simple textual context by joining retrieved chunks.

Returns a single string suitable for inclusion in prompts sent to
the LLM.
"""
docs = retrieve(question)
context = "\n]\n".join(
doc.page_content
for doc in docs
)

return context
```

Recommendation Prompt Builder

Source file: recommendation.py

```
from llm import ask_llama


def _get_condition_focus(profile: dict) -> str:
medical_history = (profile.get("medical_history") or "").lower()

respiratory_terms = ["respiratory", "asthma", "copd", "bronchitis", "emphysema", "lung disease",
"pneumonia", "oxygen", "tb"]
heart_terms = ["heart", "cardiac", "heart disease", "hypertension", "bp", "high blood pressure",
"stroke",
"arrhythmia"]
cancer_terms = ["cancer", "tumor", "malignancy"]
diabetes_terms = ["diabetes", "sugar"]

if any(term in medical_history for term in respiratory_terms):
return f"""
IMPORTANT: The user has a medical history that includes severe respiratory disease:
{profile.get('medical_history')}
Make this the main reason for the recommendation.
Prioritize policies that cover hospitalization, ICU care, oxygen therapy, emergency admission,
respiratory treatment, medicines, chronic disease support, and day-care procedures.
Explain clearly why these features are valuable for someone with this condition.
"""

if any(term in medical_history for term in heart_terms):
return f"""
IMPORTANT: The user has a medical history that includes heart-related disease:
{profile.get('medical_history')}
Make this the main reason for the recommendation.
Prioritize policies that cover hospitalization, ICU care, emergency cardiac treatment, surgery,
medicines, follow-up care, and chronic disease support.
Explain clearly why these features are valuable for someone with this condition.
"""

if any(term in medical_history for term in cancer_terms):
return f"""
IMPORTANT: The user has a medical history that includes cancer-related disease:
{profile.get('medical_history')}
Make this the main reason for the recommendation.
Prioritize policies that cover hospitalization, chemotherapy, surgery, specialist treatment,
medicines,
and critical illness support.
Explain clearly why these features are valuable for someone with this condition.
"""

if any(term in medical_history for term in diabetes_terms):
return f"""
IMPORTANT: The user has a medical history that includes diabetes:
{profile.get('medical_history')}
Make this the main reason for the recommendation.
Prioritize policies that cover hospitalization, regular treatment, medicines, diabetic care, and
chronic
disease support.
```

```
Explain clearly why these features are valuable for someone with this condition.
"""

return """
If the profile includes a serious health condition, make it central to the recommendation and
explain why
the chosen policy is suitable.
"""


def build_recommendation_prompt(profile: dict, context: str) -> str:
condition_focus = _get_condition_focus(profile)

return f"""
You are an expert Insurance Advisor.
User Profile
Name : {profile.get("name")}
Age : {profile.get("age")}
Gender : {profile.get("gender")}
Occupation : {profile.get("occupation")}
Income : {profile.get("income")}
Marital Status : {profile.get("marital_status")}
Medical History : {profile.get("medical_history")}
Existing Insurance : {profile.get("existing_policy")}

Insurance Documents
{context}

Based ONLY on the provided user profile and insurance documents,

{condition_focus}

Recommend:
1. Best insurance policy
2. Why it suits the customer
3. Coverage details
4. Premium (if available)
5. Most useful parts of the policy
6. Why this premium is valuable for the customer
7. Advantages
8. Important exclusions
9. Confidence score (0-100%)

Present the answer in a clear bullet-point format. Highlight the most useful parts of the policy
and explain
why the premium is valuable.
Do not give a generic answer. Tie every recommendation to the user's profile and medical
history.

If the document does not contain the requested information, clearly say "not available"
"""


def generate_recommendation(profile: dict, context: str):
prompt = build_recommendation_prompt(profile, context)
return ask_llama(context, prompt)
```

Shared Memory State

Source file: memory.py

```
from langchain_core.vectorstores import InMemoryVectorStore


class Memory:

def __init__(self):
self.profile ={}
self.chat_history =[]
self.vector_store = None
self.documents = []
```

```
memory=Memory()
```

Workflow Graph Fallback

Source file: graph.py

```
import os
from typing import TypedDict
from rag import get_context
from recommendation import generate_recommendation
from memory import memory # Imported instance to fetch user profile data

# 1. Keep your state layout structure using standard Python types
class GraphState(TypedDict):
question: str
context: str
answer: str

# 2. Keep your node logic exactly as you designed it
def retrieve_node(state: GraphState) -> GraphState:
# Fixed typo: changed state["questions"] to state["question"]
context = get_context(state["question"])
state["context"] = context
return state

def recommendation_node(state: GraphState) -> GraphState:
# Use memory.profile and the retrieved context to build the answer
answer = generate_recommendation(memory.profile, state["context"])
state["answer"] = answer
return state

# 3. Create a clean Python fallback class to mimic the graph compilation
class PythonWorkflowFallback:
def __init__(self):
pass

def run(self, initial_state: dict) -> dict:
"""
Executes your custom nodes sequentially instead of using LangGraph.
Matches the path: Entry -> Retrieve Node -> Recommendation Node -> Finish
"""
# Make a copy of the incoming state layout
state = initial_state.copy()

# Step 1: Run the retrieve node function
state = retrieve_node(state)

# Step 2: Run the recommendation node function
state = recommendation_node(state)

return state

# 4. Export the object as 'graph' so 'streamlit_app.py' imports it successfully
graph = PythonWorkflowFallback()
```

Pydantic Models

Source file: models.py

```
from pydantic import BaseModel

class UserProfile(BaseModel):
name:str
age:int
gender:str
occupation:str
income:float
marital_status:str
medical_history:str
```

```
existing_policy:str


class Recommendation(BaseModel):
policy_name:str
```

LLM Smoke Test

Source file: test_llm.py

```
from langchain_ollama import ChatOllama
llm=ChatOllama(model="llama3.2:latest")
response = llm.invoke("hello")
print(response.content)
```

Recommendation Prompt Tests

Source file: tests/test_recommendation_prompt.py

```
import unittest

from recommendation import build_recommendation_prompt


class RecommendationPromptTests(unittest.TestCase):
def test_build_recommendation_prompt_highlights_respiratory_condition(self):
profile = {
"name": "John",
"age": 45,
"gender": "Male",
"occupation": "Engineer",
"income": 1200000,
"marital_status": "Married",
"medical_history": "Severe respiratory disease",
"existing_policy": "No policy",
}

prompt = build_recommendation_prompt(profile, "sample document")

self.assertIn("severe respiratory disease", prompt.lower())
self.assertIn("oxygen", prompt.lower())
self.assertIn("icu", prompt.lower())
self.assertIn("hospitalization", prompt.lower())

def test_build_recommendation_prompt_highlights_heart_disease(self):
profile = {
"name": "John",
"age": 45,
"gender": "Male",
"occupation": "Engineer",
"income": 1200000,
"marital_status": "Married",
"medical_history": "Mild heart disease",
"existing_policy": "No policy",
}

prompt = build_recommendation_prompt(profile, "sample document")

self.assertIn("heart-related disease", prompt.lower())
self.assertIn("cardiac", prompt.lower())
self.assertIn("hospitalization", prompt.lower())


if __name__ == "__main__":
unittest.main()
```

Requirements

Source file: requirements.txt

```
streamlit
langchain
langchain-core
langchain-community
langchain-ollama
ollama
pypdf
python-docx
sentence-transformers
python-dotenv
tqdm
faiss-cpu
pydantic
playwright
```

9. Uploaded Test Data

Insurance SOP Test Document

Source file: uploads/HealthSecurePlus_SOP.txt

```
HealthSecure Plus - Standard Operating Procedure (SOP)

Version: 1.0

TABLE OF CONTENTS 1. Introduction 2. Product Overview 3. Eligibility 4.
Sum Insured Options 5. Coverage 6. Benefits 7. Waiting Periods 8.
Exclusions 9. Optional Riders 10. Premium Factors 11. Underwriting
Guidelines 12. Cashless Hospitalization 13. Reimbursement Claims 14.
Claim Documents 15. Renewal 16. Cancellation 17. Fraud Prevention 18.
Customer Service 19. Grievance Process 20. Tax Benefits 21. FAQs 22.
Customer Scenarios 23. Recommendation Guidelines

1. INTRODUCTION HealthSecure Plus is a comprehensive individual health
insurance product designed to provide financial protection against
hospitalization and medical expenses. The policy supports cashless
and reimbursement claims through an approved hospital network.

2. PRODUCT OVERVIEW Policy Type: Individual Health Insurance Entry Age:
18–65 years Renewal: Lifetime Coverage Options: ■3L, ■5L, ■10L,
■15L, ■25L, ■50L

3. ELIGIBILITY • Indian residents • Valid identity proof • Medical
underwriting where applicable

4. COVERAGE • Hospitalization • ICU • Room Rent • Nursing • Surgery •
Diagnostics • Ambulance • Organ Donor • AYUSH • Day Care • Modern
Treatments

5. BENEFITS • Cashless treatment • Annual health check-up • No Claim
Bonus • Restore Benefit • Teleconsultation • Health coaching

6. WAITING PERIODS General illness: 30 days Pre-existing diseases: 24
months Specified diseases: 24 months Accidents: Immediate

7. EXCLUSIONS Cosmetic surgery, alcohol/drug abuse, self-inflicted
injury, experimental treatment (unless covered), war, nuclear risks.

8. OPTIONAL RIDERS Critical Illness Personal Accident Hospital Cash
Maternity Room Rent Waiver

9. PREMIUM FACTORS Age, Sum Insured, City, Medical History, BMI,
Smoking Status, Occupation.

10. UNDERWRITING Risk assessment includes age, medical history,
occupation and previous claims.

11. CASHLESS CLAIM PROCESS Notify hospital, submit health card and ID,
pre-authorization, insurer approval, treatment, direct settlement.

12. REIMBURSEMENT CLAIM Pay hospital, collect bills, submit claim,
verification, settlement.

13. REQUIRED DOCUMENTS Policy copy ID Proof Address Proof Medical
Reports Bills Discharge Summary Cancelled Cheque

14. RENEWAL Grace Period: 30 days Lifetime renewal available.

15. CANCELLATION Free-look period as per policy terms. Refund subject to
applicable rules.

16. FRAUD PREVENTION False claims, forged documents and intentional
misrepresentation may result in claim rejection.

17. CUSTOMER SUPPORT 24x7 Toll-Free Support Email Support Online Claim
Portal

18. TAX BENEFITS Eligible under Section 80D of the Income Tax Act
```

(India), subject to applicable laws.

19. FAQ Q: Is diabetes covered? A: Yes, after the waiting period for pre-existing conditions.

Q: Does the policy cover cancer? A: Yes, subject to policy terms.

Q: Can I renew after age 65? A: Yes, lifetime renewal is available.

20. CUSTOMER SCENARIOS

- Young employee: ■5L Health + Term Insurance.
- Married couple: Family Floater.
- Senior citizen: Senior Health Plan.
- Chronic illness: Add Critical Illness Rider.

21. RECOMMENDATION GUIDELINES Recommend policies based on age, income, family size, medical history, occupation and coverage needs.

END OF DOCUMENT